condukt

Migrating Search

From PostgreSQL (`pg_trgm`) to Elasticsearch (`BM25`)

Amir Hossein Rassafi

Head of Platform at Condukt

amir-h-rassafi.github.io

October 15, 2026

The Story

1. **Problem & Goal**
   Why and how this became an initiative
2. **Search Concepts**
   Text similarity vs search
3. **Index Design**
   Recreating fuzzy-name signals in Elasticsearch
4. **Migration Design**
   Moving data to ElasticSearch
5. **Validation**
   Proving relevance before production rollout
6. **Outcome**
   Latency gains, trade-offs, and lessons

Scope: Boundaries

What this story covers

- PostgreSQL `pg_trgm` and Elasticsearch BM25.
- Stable data replica creation
- Ranking validation and production trade-offs.

What this is not

- Vector search or embeddings.
- RAG context matching or answer generation.

Problem & Goal: Initiative

- The search covers **multilingual** company names, with spelling and spacing variations.
- The current search worked and users **trusted** its **fuzzy matching** behavior.
- It ran on PostgreSQL with **16 CPU** cores and **64 GB** memory.
- At around **10M** records, **p99** latency was about **10 seconds**.
- Maximum sustainable **throughput** was around **2 requests per second**.

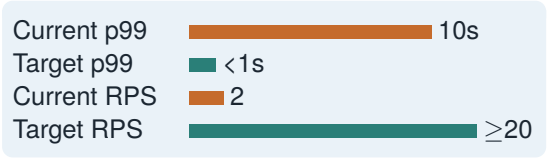
Goal

Bring **p99** search latency **under 1 second**, increase **throughput** by at least **10x**, and **preserve the ranking behavior** users already trusted.

Problem & Goal: Options

After applying the available PostgreSQL and index optimizations, the remaining options were:

- Scale up the database.
  - More CPU helped throughput and concurrency, but not single-query latency.
  - It also kept search coupled to an expensive source-of-truth database.
- Move search to a dedicated system.



Problem & Goal: Tool Investigation

Initial pool: 11 options

Typesense	Elasticsearch
Meilisearch	Faiss
pgvector	Weaviate
Redis	Milvus
Qdrant	Pinecone
Annoy	

Evaluation dimensions

- **Search:** full text, fuzzy, use-case fit.
- **Scale:** clustering and data size.
- **Operations:** ease of use, deployment.
- **PostgreSQL:** integration and replica setup.
- **Ecosystem:** community, implementation languages.
- **Future:** vector-search support.

Shortlist after filtering

Typesense, Meilisearch, and Elasticsearch: closest fit to the immediate full-text and fuzzy company-name search need.

Problem & Goal: Candidate Comparison

The three shortlisted candidates, after filtering the initial pool.

Factor	Typesense	Meilisearch	Elasticsearch
Availability	Raft-based HA cluster	Replication in Enterprise Edition	Primary shards and replicas
Scale model	Full dataset replicated on each HA node	Sharding across instances in Enterprise Edition	Index distributed across primary shards
Search control	Field weights, prefix and typo settings	Ordered ranking rules and typo settings	Custom analyzers, multi-fields, query boosts

Why Elasticsearch

Typesense and Meilisearch were strong fits. Existing Elasticsearch expertise and limited benchmark evidence for future scale tipped our decision.

Capability summary checked September 2025; sources in presenter notes. The latency target was an acceptance criterion, not a three-engine benchmark result.

Search Concepts: Why This Matters

Because PostgreSQL `pg_trgm` and Elasticsearch BM25 match and rank text differently, we need a few concepts before discussing the migration.

- Trigram similarity: how the old search matched fuzzy text.
- From character similarity to analyzed-term relevance.
- BM25: how Elasticsearch scores analyzed search terms.
- Matching tradeoffs and query fuzziness: where behavior changes.

Search Concepts: Trigram Similarity

What it is

A trigram is a rolling 3-character window. Similar strings share many windows, even when spelling or spacing differs.

- Useful for typos, spacing differences, and fuzzy names.
- Character overlap needs no language-specific stemmer.
- Normalization still matters for accents, case, and punctuation.

Why it fits

Names are short, noisy, and users usually know only an approximate spelling.

Search Concepts: Trigram Similarity Example

PostgreSQL pads each word with spaces

`search` → `{__s, _se, sea, ear, arc, rch, ch_}`

`serch` → `{__s, _se, ser, erc, rch, ch_}`

_ represents a space; every fragment has three characters.

Four shared fragments, nine distinct fragments

$$\text{similarity} = 4 / 9 \approx 0.44$$

Default `pg_trgm.similarity_threshold`: 0.3 for the % filter; higher scores match, lower scores are excluded. Configurable per session.

Search Concepts: BM25 Formula

Analyzer creates terms

`apple apple banana`

$\rightarrow$ `apple`, `apple`, `banana`

Base idea

$$\text{score} \approx tf \times idf$$

- tf: term frequency in this document.
- idf: rarity across all documents.

BM25 extension

$$\sum_{t \in q} \text{idf}(t) \cdot \frac{tf(t,d)(k_1+1)}{tf(t,d) + k_1 \left(1 - b + b \frac{|d|}{avgdl}\right)}$$

- Rare terms still matter most.
- Repeated terms help, but the gain saturates.
- Long noisy documents are normalized against average length.

Key distinction

`pg_trgm` is text similarity. BM25 scores analyzed search terms.

Search Concepts: Typo Example

```
{
  "match": {
    "name": {
      "query": "apple orangge",
      "boost": 70,
      "fuzziness": "auto"
    }
  }
}
```

Example candidate

Apple Orange

`orangge` differs from `orange` by one inserted `g`.

This is the fuzzy name clause from our query. A separate prefix clause uses `fuzziness: "1"`.

Search Concepts: Matching Tradeoffs

Similarity wins

Query: `jon smyth`

Record: `john smith`

BM25 wins

Query: `senior backend engineer`

Record: `backend engineer with search experience`

With simple word / space tokenization

Length-sensitive similarity: one- and two-character input

Candidate	Query `a`	Query `ab`
`ab`	0.25	1.00
`abc`	0.20	0.40
`abcd`	0.17	0.33
`abcde`	0.14	0.29

Every query is a perfect prefix. Additional candidate trigrams dilute whole-string similarity.

Match: score ≥ 0.3

No match: score < 0.3

Controlled examples; scores rounded.

Search Concepts: From Similarity to Relevance

PostgreSQL: character overlap

How similar are these two names?

1. Compare their trigram sets.
2. Keep names above the similarity cutoff.
3. Order by similarity.

Typos can still share many character fragments.

Elasticsearch: analyzed terms

Which matching names are most relevant?

1. Analyze names and query text into terms.
2. Match terms; optionally expand for typos.
3. Rank text matches with BM25 and boosts.

BM25 scores matches; fuzziness handles term spelling differences.

Index Design: Country-Specific Indexes

- We maintain a separate search index for each indexed country.
- Languages and search settings differ between countries.
- The overall multi-field analyzer approach stays similar.

Index Design: Elasticsearch Multi-fields

```
"name": {
  "type": "text", "analyzer": "language_text_standard",
  "fields": {
    "normalized_keyword": {
      "type": "keyword", "normalizer": "language_normalizer"
    },
    "prefix": {
      "type": "text", "analyzer": "prefix_analyzer",
      "search_analyzer": "keyword_normalized"
    },
    "whitespace_tokenizer": {
      "type": "text", "analyzer": "language_text_whitespace"
    },
    "word_shingle": {
      "type": "text", "analyzer": "word_shingle_analyzer"
    }
  }
}
```

Excerpt from the supplied default mapping; other subfields omitted.

Index Design: Punctuation Changes Tokens

Tokenizer	Input: `a-m`	Input: `a m`
Standard	`[a, m]`	`[a, m]`
Whitespace	`[a-m]`	`[a, m]`
Keyword	`[a-m]`	`[a m]`

Preserve the useful distinction

Standard tokenization merges these two inputs into the same terms. Whitespace and keyword representations retain their different forms.

Tokenizer-only examples; production char filters and token filters can change the result. Verify the full chain with `_analyze`.

Index Design: Why Separate Prefixes

- Analyzer behavior on names such as `a-m` and `a m` drove the design.
- Separate fields retain different token boundaries and matching signals.
- Prefix matching has its own weighting, with and without fuzziness.

Why not `search_as_you_type`?

We needed control over these representations. That field type also accepts custom analysis; this example explains our multi-field choice, not a universal limitation of the alternative.

Index Design: Word Shingles

Input words, in order

`apple` → `pear` → `plum`

Adjacent word shingles

2 words `apple pear` `pear plum`

3 words `apple pear plum`

- Shingles join adjacent tokens in their original order.
- They add phrase signals alongside individual words.
- They are a text-analysis technique, unrelated to `pgvector`.

Index Design: Our Search Query

```
{"query": {"bool": {
    "should": [
        {"term": {"name.normalized_keyword":
            {"value": "SEARCH_TEXT", "boost": 300}}},
        {"match": {"name.prefix":
            {"query": "SEARCH_TEXT", "boost": 40}}},
        {"match": {"name.prefix":
            {"query": "SEARCH_TEXT", "boost": 40, "fuzziness": "1"}}},
        {"match": {"name":
            {"query": "SEARCH_TEXT", "boost": 70, "fuzziness": "auto"}}},
        {"match": {"name.whitespace_tokenizer":
            {"query": "SEARCH_TEXT", "boost": 30}}},
        {"match": {"name.word_shingle":
            {"query": "SEARCH_TEXT", "boost": 10}}}
    ],
    "minimum_should_match": 1
}}}
```

Replace `SEARCH_TEXT` with the same input in all six clauses. At least one clause must match.

Index Design: Analyzer Choices

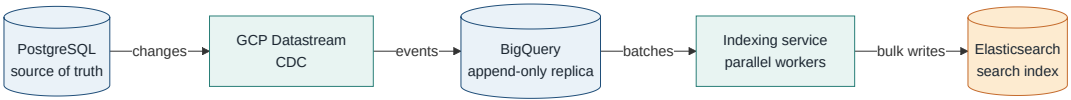
- Normalize case and accents with lowercase + ASCII folding.
- Main text: standard tokenizer with country-specific analysis and stemming.
- Prefix: keyword tokenizer, edge n-grams 3–20; keyword query analysis.
- Whitespace: separate word boundaries and a legal-suffix protection filter.
- Shingles: whitespace tokens, 2–3 words, plus individual words.

Underlined components link to Elastic documentation; analyzer names and their combination are ours.

Migration Design: Requirements

- **Deploy** Elasticsearch **quickly** with low operational overhead.
- **Replicate** required PostgreSQL data into Elasticsearch.
- **Benchmark old** and **new** ranking behavior.
- **Prove** the new path before production rollout.

Migration Design: Data Path



PostgreSQL owns the data; BigQuery holds change history; Elasticsearch serves search.

Migration Design: Elasticsearch Deployment

- Deployment: managed by Elastic **Cloud Hosted**.
- Cluster: 3 data nodes, each with **4 CPU** cores and **8 GB RAM**“““.
- Default country index: **3 primary shards**.
- Replicas per primary: **1**
- Refresh interval: **300 seconds (5 minutes)**.
- Elasticsearch handles shard allocation, routing, and replication.

Why Cloud Hosted

We chose Elastic Cloud Hosted for greater control over index settings, shards, and replicas.

Migration Design: Document Identity

- Reuse the PostgreSQL row ID as the Elasticsearch document ID.
- Trace the same entity across the database, CDC, and search.
- Repeating a document replacement avoids duplicate documents.

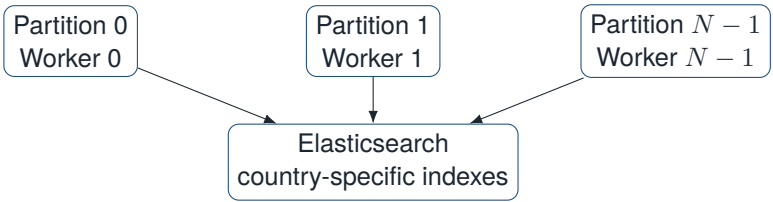
Replay also needs ordering

A stable ID alone cannot stop an older change overwriting a newer one. Preserve source order or enforce source versions, including deletes.

Migration Design: Indexer Partitions

Deterministic assignment

```
partition = hash(entity_unique_id) % N
```



Our indexer used one worker per partition: a 1:1 assignment.
Indexer partitions (application shards) are separate from Elasticsearch index shards.

Migration Design: Checkpointing

One partition's ordered batch

Read checkpoint → fetch rows → bulk write

Check every item → commit the partition checkpoint

- Cursor: source timestamp, source sequence / LSN, document ID.
- Advance only past successfully handled events.
- On failure, retry from the last committed checkpoint.

An HTTP success response does not mean every bulk item succeeded.

Validation: Generated Queries with NLPAUG

Use NLPAUG and search-specific transformations to generate:

- Keyboard-adjacent and OCR-like substitutions.
- Character insertion, substitution, swapping, and deletion.
- Word swapping, cropping, deletion, and splitting.
- Spacing and casing variations.
- Keyword slicing and query truncation: partial names or dropped trailing words.

Compare every generated query across both search setups; then validate historical queries through expert review of both top-20 lists.

NLPAUG provides the noise augmenters; keyword slicing/query truncation is our search-specific test generation.

Validation: What MRR Measures

Mean Reciprocal Rank: average $\frac{1}{\mathbf{rank}}$ of the first relevant result.

Query	First relevant result	Reciprocal rank
A	Rank 1	1
B	Rank 2	0.5

Standard MRR@20 (illustrative)

$(1 + 0.5)/2 = 0.75$

Higher is better; use the same queries, relevance labels, and cutoff.

The next slide reports found-only MRR; misses are listed separately.

Validation: MRR Results(Generated Test Cases)

Search system	MRR (found only)	Not found(within first 20)
PostgreSQL `pg_trgm`	0.95	26%
Elasticsearch	0.91	14%

Validation: Backtesting

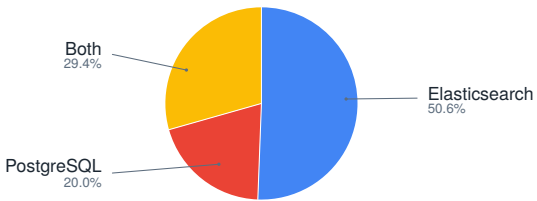
1. Take historical customer search queries.
2. Run each query against PostgreSQL and Elasticsearch.
3. Share both ordered lists of 20 results with domain experts.
4. Ask experts to vote for the list they would expect to see.

What the votes measure

Expert preference between result lists, not customer funnel selections. List-level votes are not MRR relevance labels.

Validation: Human Judgement

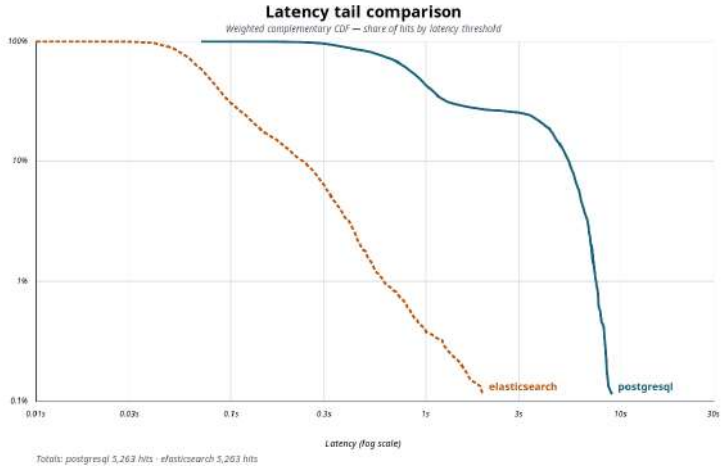
- Same historical query; two top-20 lists.
- Domain experts vote for the expected result list.
- `Both`: both lists are acceptable.



Decision signal

Elasticsearch was preferred or tied in 80% of reviewed cases.

Outcome: Latency Comparison



Thank you

Amir Hossein Rassafi

Head of Platform at Condukt

amir-h-rassafi.github.io